

Proyecto Andes — Convex Interpretable Clustering System

(Documento Completo de Industrialización)

Desglose del Proyecto

El Convex Interpretable Clustering System es una librería Python de producción que implementa clustering convexo interpretable mediante los algoritmos ADMM, AMA y Douglas-Rachford (DR), acompañada de un pipeline de regresión regularizada (Lasso y Boosting) y un dashboard interactivo deployado en Streamlit Cloud. El objetivo es transformar el código académico de la Universidad de los Andes en un sistema industrial completo: testado, empaquetado, comparado contra benchmarks estándar y deployado con una API REST consumible.

Componentes de Industrialización

1. Fundamentos del Código

El primer paso antes de añadir cualquier capa de infraestructura es limpiar el código base. Un código con paths hardcoded, funciones sin estructura y variables mal nombradas no puede integrarse en CI/CD ni escalarse a la nube sin errores.

1.1 Eliminar los paths hardcoded

— Reemplazar todas las rutas absolutas por rutas relativas calculadas con `pathlib.Path(__file__).parent` o por variables de entorno (p.ej. `os.environ['DATA_DIR']`).

Por qué pathlib: Es la forma idiomática en Python 3.6+ de manejar rutas. Es cross-platform (funciona en Windows, Linux y macOS sin cambios), no requiere strings manuales con '/' o '\\', y permite construir rutas de forma composable. Los paths hardcoded son el error más común que rompe la reproducibilidad y el CI.

1.2 Centralizar la carga de datasets

— Crear una función `load_dataset(name)` que busque el dataset localmente y, si no existe, lo descargue desde internet (S3 o URL pública) automáticamente.

Por qué centralizar: Sin una función única de carga, cada script tiene su propia lógica de descarga, produciendo duplicados y rutas inconsistentes. Una función central permite además cachear datasets, agregar logging de descarga y cambiar la fuente (local → S3) en un solo lugar.

1.3 Consolidar utilidades con `utils.py`

— Mover todas las funciones reutilizadas en múltiples módulos a un archivo `src/utils.py`.

Por qué utils.py: Evita la duplicación de código (DRY principle). Cualquier bug en una función compartida se corrige en un solo lugar. Es la convención estándar en proyectos Python medianos y facilita el testing unitario de funciones auxiliares.

1.4 Corregir nomenclatura de variables y funciones

— Seguir PEP 8: `snake_case` para variables y funciones, `PascalCase` para clases. Renombrar variables cortas o ambiguas.

Por qué PEP 8: Es el estándar de facto en Python. Herramientas como `ruff`, `mypy` y `pylint` lo esperan. El código que no lo sigue genera warnings automáticos en cualquier CI moderno y dificulta la lectura por parte de reclutadores o revisores técnicos.

2. Testing

Un proyecto sin tests no es reproducible. Los tests de regresión garantizan que los algoritmos núcleo no se rompan silenciosamente entre commits.

2.1 Test de convergencia de los 6 puntos

— Crear un test con `pytest` que ejecute ADMM, AMA y DR sobre el toy dataset de 6 puntos y verifique que los centros convergen a los centros conocidos analíticamente.

— Usar `evaluation()` como assertion: si el MSE supera un umbral predefinido, el test falla y el pipeline de CI se detiene.

Por qué `pytest`: Es el framework de testing estándar en Python moderno. Ofrece fixtures reutilizables, parametrización de tests y output legible. Los tests numéricos de convergencia son el equivalente en ML de los unit tests en software: garantizan que la matemática sigue siendo correcta tras cada refactor.

2.2 Test de equivalencia `fast_RFS` vs `R_FS`

— Verificar que los coeficientes producidos por `fast_RFS` y `R_FS` sean iguales dentro de tolerancia numérica (`np.allclose(coef_fast, coef_standard, atol=1e-6)`).

Por qué este test: Si una versión acelerada produce resultados distintos a la versión de referencia, hay un bug silencioso. Este test de equivalencia es la garantía matemática de que la optimización no introdujo errores.

3. CI/CD — GitHub Actions

El CI/CD convierte los tests en guardianes automáticos del repositorio. Cada push dispara una verificación completa sin intervención manual.

3.1 Action de regresión numérica

— GitHub Action que, en cada push a `main` o `develop`, ejecuta los siguientes tests sobre el toy dataset de 6 puntos:

— Tests de convergencia de clustering: ADMM, AMA y DR verifican que los centros convergen a los centros conocidos del dataset de 6 puntos.

— Tests de coeficientes de regresión: `RFS`, `FAST RFS` y sus dos versiones alternativas (`fast_RFS_sparse` y `R_FS_alt`) verifican que los coeficientes producidos son correctos mediante MSE sobre el dataset de 6 puntos. El MSE es la métrica natural aquí porque estos algoritmos resuelven un problema de regresión lineal regularizada.

— Si cualquiera de los tests falla (MSE de regresión supera umbral, o centros de clustering no convergen), el pipeline se detiene.

Por qué GitHub Actions: Es la plataforma CI nativa de GitHub, sin coste para repositorios públicos. No requiere configurar un servidor externo (Jenkins, CircleCI). El workflow se define en un YAML versionado dentro del repo, lo que hace el CI completamente reproducible.

3.2 Action de estilo con ruff

— Action separado que ejecuta `ruff check` y falla si hay errores de estilo o imports no usados.

Por qué ruff y no flake8/pylint: ruff es un linter escrito en Rust que es 10–100× más rápido que flake8. Cubre PEP 8, imports, complejidad ciclomática y más, todo en una sola herramienta. Es el estándar emergente en proyectos Python modernos (2023+).

3.3 Badge de CI en el README

— Añadir el badge de estado de GitHub Actions al README.md para que sea visible que los experimentos son reproducibles.

Por qué el badge: Un badge verde de CI en el README es una señal inmediata de calidad industrial. Cualquier reclutador o colaborador que abra el repositorio sabe en dos segundos que el código tiene tests y que pasan. Es el equivalente visual de un certificado de calidad.

4. Cloud — AWS S3 + Google Cloud Run

Mover los datos y los jobs de computación pesada a la nube transforma el proyecto de un script local en un sistema distribuido y reproducible.

4.1 Datasets en S3

— Estandarizar la carga de datos: todos los datasets se leen desde un bucket S3 usando boto3. La función `load_dataset(name)` descarga al vuelo si no hay caché local.

Por qué S3: Es el estándar de almacenamiento de objetos en la industria. Los buckets S3 son accesibles desde cualquier máquina (local, CI, cloud run) sin configurar un servidor. El tier gratuito de AWS cubre los datasets pequeños de este proyecto. boto3 es el SDK oficial de Python para AWS, con manejo de credenciales, reintentos y streaming de archivos grandes.

4.2 test_boosting como job paralelo en Google Cloud Run

— Empaquetar `test_boosting.py` como un job en Google Cloud Run (o AWS Batch) que ejecute los experimentos en paralelo y guarde los resultados en el bucket S3 en formato CSV.

Por qué Cloud Run (Jobs): Google Cloud Run Jobs está diseñado exactamente para este patrón: ejecutar un script por demanda, sin mantener un servidor activo. Se lanza con un solo comando, escala horizontalmente (múltiples jobs paralelos = múltiples experimentos simultáneos), y cobra solo por tiempo de cómputo efectivo. Es una opción más simple que Kubernetes para experimentos one-shot.

4.3 fast_rfs_sparse y algoritmos compatibles en cloud

— Containerizar `fast_rfs_sparse` con Docker y deployarlo como job en Google Cloud Run para ejecutar experimentos de gran escala sin bloquear la máquina local.

Por qué Docker para el cloud job: Docker garantiza que el entorno de ejecución en la nube es idéntico al local. Sin Docker, las diferencias de versiones de librerías entre la máquina del desarrollador y el runner del cloud pueden producir resultados distintos — el problema clásico de 'en mi máquina funciona'.

5. Empaquetado como Librería Python

Convertir el proyecto en un paquete Python instalable es el paso que lo hace consumible por terceros y comparable con librerías de referencia como scikit-learn.

5.1 pyproject.toml

— Crear `pyproject.toml` con metadata del paquete (nombre, versión, autor, dependencias), siguiendo PEP 621. Usar `hatchling` o `setuptools` como build backend.

Por qué pyproject.toml y no setup.py: pyproject.toml es el estándar moderno (PEP 517/518/621) que reemplaza setup.py. Permite definir el build backend, las dependencias de desarrollo y la configuración de herramientas (ruff, mypy, pytest) en un único archivo. setup.py está en modo deprecación en el ecosistema Python actual.

5.2 Exportaciones públicas en `__init__.py`

— Definir en `src/convex_clustering/__init__.py` las exportaciones públicas del paquete: `ConvexClusterer`, `ADMM`, `AMA`, `DR`, `Boosting`.

Por qué __init__.py explícito: Sin él, el usuario tendría que importar desde submódulos internos (frágil ante refactors). Con él, la API pública es estable: from convex_clustering import ConvexClusterer funciona independientemente de cómo esté organizado el código internamente.

5.3 Semantic Versioning + CHANGELOG

— Versionar el paquete con `semver`: `MAJOR.MINOR.PATCH`. Mantener un `CHANGELOG.md` con los cambios por versión.

5.4 `fit()` y `predict()` en Boosting

— Implementar los métodos `fit(X, y)` y `predict(X)` en la clase `Boosting` para que sea compatible con la API de scikit-learn.

Por qué la API de sklearn: Scikit-learn define la interfaz estándar de estimadores en Python ML. Implementar fit/predict permite a cualquier usuario reemplazar sklearn's GradientBoostingRegressor por tu Boosting sin cambiar el código circundante. Es la señal más fuerte de que el código es production-ready.

6. Compatibilidad con `sklearn.pipeline.Pipeline`

Hacer `ConvexClusterer` compatible con el `Pipeline` de scikit-learn es una señal técnica muy fuerte: significa que el algoritmo puede integrarse en cualquier workflow de ML estándar sin adaptaciones.

6.1 Implementar `BaseEstimator` y `ClusterMixin`

— Heredar de `sklearn.base.BaseEstimator` y `sklearn.base.ClusterMixin` en la clase `ConvexClusterer`. Implementar `fit(X)` y `fit_predict(X)`.

— Esto permite el uso directo en `Pipeline`:

```
Pipeline([('scaler', StandardScaler()), ('clusterer', ConvexClusterer(gamma=0.5))])
```

Por qué BaseEstimator: BaseEstimator regala gratis los métodos `get_params()` y `set_params()`, necesarios para `GridSearchCV` y `RandomizedSearchCV`. ClusterMixin regala `fit_predict()`. La compatibilidad con `Pipeline` es lo que diferencia una implementación de investigación de una librería industrial. También permite usar el clusterer en herramientas como `mlflow autologging` o `SHAP` directamente.

7. Type Annotations + mypy

7.1 Type hints completos en todas las funciones públicas

— Añadir type hints a todas las funciones públicas del paquete. Ejemplo:

```
def fit(self, X: np.ndarray, gamma: float = 0.5) -> 'ConvexClusterer': ...
```

— Integrar mypy --strict como paso de validación en el CI.

Por qué mypy: Los type hints se añaden en horas pero su impacto visual en el código es enorme: comunican la interfaz esperada sin leer el docstring, permiten autocompletado en IDEs (VSCode, PyCharm), y los errores de tipo se detectan antes de ejecutar el código. mypy es el type checker estándar de Python, usado en producción por Google, Meta y Dropbox.

8. FastAPI — Endpoint REST

Una API REST hace al algoritmo consumible desde cualquier lenguaje o frontend, desacoplando el modelo del dashboard de visualización.

8.1 Endpoint POST /cluster

— Endpoint POST /cluster que recibe un dataset (JSON con matriz X) y los hiperparámetros (gamma, epsilon, num_iter, algorithm) y devuelve los cluster paths, los centros finales y la curva de convergencia.

— Endpoint GET /algorithms que lista los algoritmos disponibles y sus parámetros.

— Endpoint POST /compare que recibe un dataset y devuelve la comparación entre ConvexClusterer, KMeans y DBSCAN.

Por qué FastAPI y no Flask: FastAPI genera documentación Swagger/OpenAPI automática (disponible en /docs), valida los tipos de entrada con Pydantic, y es asíncrono por defecto. Es 2–3× más rápido que Flask en benchmarks de requests/segundo. El Swagger automático es especialmente valioso para mostrar en el CV: cualquier reclutador puede probar los endpoints sin escribir código.

8.2 Docker + docker-compose

— Un docker-compose.yml que levante la API (FastAPI) y el dashboard (Streamlit) juntos con un solo docker-compose up.

Por qué docker-compose: Elimina la necesidad de lanzar dos procesos manualmente en terminales separadas. Define la relación entre servicios (el dashboard llama a la API), las variables de entorno y los puertos. Es el estándar para desarrollo local de sistemas multi-servicio.

9. Dashboard de Visualización — Streamlit

El dashboard convierte el algoritmo en un producto interactivo. Transforma una función de Python en una herramienta que cualquier analista puede usar sin escribir código.

9.1 Tab 1 — Clustering Interactivo (reemplaza `animation()`)

- Convertir `animation()` en un dashboard Streamlit con:
 - Selección del dataset (toy de 6 puntos, blobs, moons, circles, o upload CSV).
 - Selección del algoritmo: ADMM, AMA, DR, RFS, FAST RFS, y las dos versiones alternativas de RFS y FAST RFS.
 - Sliders para los parámetros: `gamma`, `epsilon`, `num_iter`.
 - Visualización de los cluster paths con Plotly (líneas de trayectoria de cada punto).

Por qué Streamlit y no Dash: Streamlit tiene una curva de aprendizaje notablemente menor que Dash. Una aplicación funcional se escribe en ~50 líneas. Tiene deployment nativo en Streamlit Cloud (gratis para repos públicos) sin configurar servidores. Para mostrar un proyecto en el CV, Streamlit permite compartir una URL pública en minutos.

9.2 Tab 2 — Comparación Lasso vs Boosting

- Tabla interactiva con Plotly que muestre R^2 , MSE y coeficientes no nulos para Lasso y Boosting en cada dataset disponible.
- Gráfico de barras comparativo de coeficientes seleccionados por cada método.

Por qué Plotly: Plotly genera gráficas interactivas (hover, zoom, pan) que funcionan directamente en Streamlit sin configuración extra. Sus gráficas son de calidad publicación. Alternativas como matplotlib generan imágenes estáticas, que son menos atractivas en un dashboard.

9.3 Tab 3 — Visualización del Grafo de Similitud

- Integrar la visualización del grafo en el dashboard: selector entre grafo fully-connected y KNN (k ajustable con slider), y visualización de cómo cambia el grafo y los clusters resultantes.

Por qué visualizar el grafo: El grafo de similitud es el componente central del clustering convexo que lo diferencia de KMeans. Visualizarlo interactivamente permite demostrar la interpretabilidad del método — su principal ventaja competitiva. Se puede usar PyVis o networkx + Plotly para el grafo.

9.4 Deployment en Streamlit Cloud

- Deployar la app en Streamlit Cloud con datasets pequeños (toy data, blobs, moons). Tener una URL pública listable en el CV.

10. DVC — Data Version Control

10.1 Trazabilidad de experimentos con DVC

- Los datasets están en S3, pero sin DVC no hay trazabilidad de qué versión de datos generó qué resultados. Añadir DVC al repositorio y conectarlo al bucket S3.
- Añadir al README: `dvc pull && dvc repro` para reproducir todos los experimentos.

Por qué DVC y no solo S3: S3 almacena los datos, pero DVC versiona la relación entre código, datos y resultados. Es el Git de los datos en ML. Sin DVC, no puedes responder '¿qué dataset y qué versión del código generaron este resultado?'. DVC también permite comparar métricas entre experimentos (`dvc metrics`).

show) sin MLflow.

11. Observabilidad del Algoritmo

Instrumentar el algoritmo para que sus métricas internas sean inspeccionables es lo que diferencia un algoritmo de investigación de uno listo para producción.

11.1 Curva de convergencia como artefacto CSV

— Exportar la curva de convergencia (loss vs iteración) de todos los algoritmos — ADMM, AMA, DR, RFS, FAST RFS y sus dos versiones alternativas — a un archivo CSV al final de cada ejecución.

11.2 MLflow / Weights & Biases para tracking

— Integrar MLflow (local) o W&B; (cloud) para loggear cada experimento con sus hiperparámetros (gamma, epsilon, num_iter), métricas finales según el tipo de algoritmo: MSE (si aplica — en los algoritmos de regresión RFS y FAST RFS que resuelven clustering vía regresión lineal regularizada) y Silhouette Score (métrica de clustering válida para todos los algoritmos), además de los artefactos (curva de convergencia, cluster assignments).

Por qué MLflow y no solo print: MLflow persiste los resultados en una base de datos local consultable. Permite comparar runs con distintos hiperparámetros en una UI web (mlflow ui). Es el estándar open-source para experiment tracking en ML, usado en producción en Databricks, Azure ML y AWS SageMaker. Un CV que menciona MLflow señala experiencia con workflows de ML industrial.

11.3 Métricas de uso de memoria

— Añadir profiling de memoria con `tracemalloc` o `memory_profiler` para reportar el peak de memoria de cada algoritmo.

Por qué medir memoria: Los algoritmos de clustering convexo pueden ser costosos en memoria para datasets grandes. Reportar el uso de memoria junto al tiempo de ejecución es la diferencia entre un benchmark superficial y un análisis riguroso.

12. Reproducibilidad

12.1 Fijar semillas en funciones con aleatoriedad

— Añadir `np.random.seed(seed)` o `random.seed(seed)` en cada función que use aleatoriedad. Exponer seed como parámetro.

12.2 `run_experiments.sh`

— Crear un script Bash `run_experiments.sh` que reproduzca todos los experimentos del paper/proyecto con un solo comando.

Por qué un script Bash: Un README que diga 'ejecuta run_experiments.sh para reproducir todo' es mucho más convincente que una lista de comandos. Es el estándar en repositorios de papers de ML (NeurIPS, ICML).

12.3 `environment.yml` / `requirements.txt`

— Crear un `requirements.txt` con versiones pinadas y un `environment.yml` para conda. El CI debe instalar exactamente estas versiones.

13. Gestión de Experimentos

13.1 Centralizar nombres de experimentos

- Crear una función `make_experiment_id(algorithm, dataset, params)` que genere un identificador único y reproducible para cada experimento.

13.2 Estructura de salida consistente

- Estandarizar el directorio de resultados:

```
results/  
{exp_id}/  
config.json ← hiperparámetros  
metrics.csv ← MSE, silhouette, tiempo  
cluster_paths.npy ← trayectorias  
convergence.csv ← curva de convergencia
```

Por qué esta estructura: Sin una estructura de salida consistente, comparar dos experimentos requiere leer el código para saber dónde guardó cada cosa. Con esta estructura, cualquier script puede cargar los resultados de cualquier experimento pasado con `results/{exp_id}/metrics.csv` sin asumir nada.

14. Benchmark Honesto — Convex vs KMeans / DBSCAN

El benchmark es el argumento de venta del algoritmo. No se trata de ganar siempre, sino de mostrar cuándo el clustering convexo es superior y por qué.

14.1 Comparación en datasets estándar de sklearn

- Sección en el notebook de demostración que compara `ConvexClusterer` vs `sklearn.cluster.KMeans` y `sklearn.cluster.DBSCAN` en:
 - `make_blobs`: clusters convexos bien separados.
 - `make_moons`: clusters no convexos (ventaja de DBSCAN).
 - `make_circles`: clusters anidados (ventaja de DBSCAN).

Por qué incluir casos donde `ConvexClusterer` no gana: Un benchmark honesto es más creíble que uno que solo muestra los casos favorables. Mostrar cuándo `KMeans` y `DBSCAN` superan al clustering convexo, y explicar por qué, demuestra comprensión profunda del espacio del problema. Además, la fortaleza específica del clustering convexo — los cluster paths y la interpretabilidad del proceso de fusión — no tiene equivalente en `KMeans` ni `DBSCAN`, y eso es lo que el benchmark debe destacar.

14.2 Métricas de comparación

- Silhouette Score, Davies-Bouldin Index, Adjusted Rand Index (si hay labels).
- Tiempo de ejecución y memoria pico.
- Interpretabilidad del path (única para `ConvexClusterer`).

15. Documentación

15.1 Docstrings en todas las funciones y clases públicas

— Formato NumPy docstring para todas las funciones públicas: descripción, parámetros (tipo + descripción), retorno, ejemplos.

Por qué NumPy docstring: Es el estándar en el ecosistema científico de Python (numpy, scipy, sklearn). Sphinx y makedocstrings lo parsean automáticamente para generar documentación HTML. Es el formato que los data scientists esperan ver.

15.2 Documentación del proyecto con MkDocs

— Generar documentación HTML con MkDocs + makedocstrings. Deployar en GitHub Pages.

Por qué MkDocs y no Sphinx: MkDocs usa Markdown puro (más simple que RST de Sphinx), tiene themes modernos (Material for MkDocs) y el deployment a GitHub Pages es un solo comando. Para un proyecto de CV, una página de documentación bonita con URL pública (user.github.io/convex-clustering) es un diferenciador visual importante.

15.3 Notebook de demostración

— Un Jupyter Notebook demo .ipynb que muestre, paso a paso:

- Instalación del paquete desde PyPI (o git).
- Uso básico con ConvexClusterer en toy data.
- Visualización de cluster paths.
- Benchmark vs KMeans/DBSCAN.
- Aplicación industrial: segmentación de clientes.
- Uso en Pipeline de sklearn.

16. Comparación contra Benchmarks del Ecosistema

16.1 Contexto en el ecosistema de clustering convexo

— Añadir una sección en el README y en el notebook que contextualice el proyecto dentro de las implementaciones existentes de clustering convexo (cvxclustr, Clusterpath R, etc.).

— Tabla comparando tiempo de ejecución y calidad de clusters en datasets de referencia.

Por qué esta comparación: Mostrar que entiendes el ecosistema de tu problema es un diferenciador muy fuerte en una entrevista técnica. Significa que el proyecto no existe en un vacío académico, sino que el desarrollador sabe cómo se posiciona respecto a las alternativas disponibles.

17. Aplicaciones Industriales Adicionales

Añadir aplicaciones concretas convierte el notebook de demostración en un argumento de venta hacia reclutadores del sector industria.

17.1 Segmentación de clientes

— Aplicar ConvexClusterer a un dataset de clientes (RFM: Recency, Frequency, Monetary). Interpretar los cluster paths como la evolución del proceso de segmentación.

17.2 Otras aplicaciones industriales de clustering

- Segmentación de documentos / topics (NLP).
- Agrupación de series temporales de sensores (IoT / manufactura).
- Cada aplicación puede ser un módulo independiente de la librería con su propio notebook de demo.

Por qué aplicaciones industriales: Los reclutadores del mercado colombiano ML/DS buscan señales de que el candidato sabe transferir técnicas académicas a problemas reales. Un notebook que segmenta clientes con tu propio algoritmo es más convincente que uno que solo muestra datasets sintéticos.

Tecnologías por Componente

Componente	Tecnologías
Fundamentos	pathlib, os.environ, PEP 8, ruff
Testing	pytest, numpy (allclose), pytest-cov
CI/CD	GitHub Actions, ruff, pytest, badges
Cloud — Storage	AWS S3, boto3, DVC
Cloud — Compute	Google Cloud Run Jobs, Docker
Empaquetado	pyproject.toml, hatchling, pip, twine
sklearn compat.	BaseEstimator, ClusterMixin, Pipeline
Type checking	type hints, mypy
FastAPI	FastAPI, Pydantic, Uvicorn, Swagger/OpenAPI
Containerización	Docker, docker-compose
Dashboard	Streamlit, Plotly, PyVis, networkx
Observabilidad	MLflow, tracemalloc, CSV artefacts
Reproducibilidad	numpy seed, requirements.txt, env.yml, DVC
Gestión Experimentos	MLflow, results/ estructura estándar
Benchmark	sklearn (KMeans, DBSCAN), silhouette, Davies-Bouldin
Documentación	MkDocs, mkdocstrings, NumPy docstrings, Jupyter
Deployment	Streamlit Cloud, GitHub Pages, Railway (opcional)

Arquitectura Completa

Librería Python (core)

- Python, pyproject.toml, src/convex_clustering/.
- Algoritmos: ADMM, AMA, Douglas-Rachford.
- Modelos de regresión: Lasso, Boosting (sklearn-compatible).
- sklearn Pipeline compatible: BaseEstimator + ClusterMixin.

Storage y Datos

- AWS S3 como repositorio de datasets y resultados.
- DVC para versionado de datos y trazabilidad de experimentos.
- results/{exp_id}/ como estructura de salida estándar.

API y Servicios

- FastAPI: endpoints /cluster, /algorithms, /compare.
- Documentación automática Swagger en /docs.
- Docker + docker-compose para levantar API + Dashboard juntos.

Observabilidad y Tracking

- MLflow para tracking de hiperparámetros y métricas.
- Curvas de convergencia exportadas como CSV.
- Métricas de memoria con tracemalloc.

Frontend / Dashboard

- Streamlit: 3 tabs (clustering interactivo, Lasso vs Boosting, grafo).
- Plotly para gráficas interactivas, PyVis/networkx para el grafo.
- Deployment en Streamlit Cloud (URL pública).

CI/CD e Infraestructura

- GitHub Actions: regression tests + ruff style check.
- Google Cloud Run Jobs para experimentos paralelos.
- GitHub Pages para documentación MkDocs.

Flujo del Sistema

- **1. Datos:** load_dataset() → descarga desde S3 si no existe localmente → datos en numpy arrays.
- **2. Core algorithm:** ConvexClusterer(algorithm='ADMM', gamma=0.5).fit(X) → cluster paths + centros.
- **3. Observabilidad:** MLflow logea hiperparámetros + métricas → curva de convergencia → results/{exp_id}/.
- **4. API:** FastAPI POST /cluster → llama al core → devuelve JSON con paths y métricas.
- **5. Dashboard:** Streamlit llama a la API → visualiza paths (Plotly) + grafo (PyVis) + benchmarks.
- **6. CI/CD:** Push → GitHub Actions: pytest (regression) + ruff (style) → badge en README.
- **7. Cloud:** Cloud Run Job: lanza batch de experimentos → guarda CSV en S3 → DVC trackea.

Esquema de Implementación

1. Refactor fundamentos: eliminar paths hardcodeados, añadir utils.py, fijar nomenclatura.
2. Añadir type hints completos y validar con mypy.
3. Escribir tests pytest: convergencia 6 puntos, equivalencia fast_RFS vs R_FS.
4. Configurar GitHub Actions: regression tests + ruff check + badge CI.
5. Crear pyproject.toml, __init__.py con exports, semantic versioning.
6. Implementar BaseEstimator + ClusterMixin en ConvexClusterer (sklearn Pipeline compatible).
7. Implementar fit() + predict() en Boosting.
8. Mover datasets a S3 con boto3; integrar DVC para trazabilidad.
9. Configurar MLflow para tracking de experimentos; exportar curvas de convergencia.
10. Estandarizar estructura de salida: results/{exp_id}/config + metrics + paths.

11. Crear `run_experiments.sh` y `environment.yml/requirements.txt`.
12. Desarrollar FastAPI: endpoints `/cluster`, `/algorithms`, `/compare` con Pydantic.
13. Crear `docker-compose.yml` para levantar API + Dashboard juntos.
14. Construir dashboard Streamlit: Tab clustering, Tab Lasso vs Boosting, Tab grafo.
15. Deployar en Streamlit Cloud (URL pública para el CV).
16. Empaquetar job `test_boosting` en Google Cloud Run para experimentos paralelos.
17. Escribir notebook de demostración: uso básico + benchmark + segmentación de clientes.
18. Generar documentación con MkDocs + `mkdocstrings`; deployar en GitHub Pages.
19. Añadir sección de benchmark honesto: ConvexClusterer vs KMeans vs DBSCAN.
20. Añadir aplicaciones industriales como módulos opcionales de la librería.